Testare functionala

Testare functionala

- Datele de test sunt generate pe baza **specificatiei** (cerintelor) programului, structura programului **nejucand nici un rol**
- Tipul de specificatie ideal pentru testarea functionala este alcatuit din **pre-conditii si post-conditii**
- Majoritatea metodelor functionale se bazeaza pe o **partitionare a datelor de intrare** astfel incat datele apartinand unei aceeasi partitii vor avea **proprietati similare (identice)** in raport cu comportamentul specificat

Partitionare de echivalenta (equivalence partitioning)

- Ideea de baza este de a partitiona domeniul problemei (datele de intrare) in partitii de echivalenta sau **clase de echivalenta** astfel incat, din punctul de vedere al specificatiei datele dintr-o clasa sunt **tratate in mod identic**.
- Cum toate valorile dintr-o clasa au specificat acelasi comportament, se poate presupune ca toate valorile dintr-o clasa vor fi procesate in acelasi fel, fiind deci **suficient sa se aleaga cate o valoare din fiecare clasa**
- In plus, domeniul de iesire va fi tratat in acelasi fel, iar clasele rezultate vor fi transformate in sens invers (**reverse engineering**) in clase ale domeniului de intrare

Partitionare de echivalenta (equivalence partitioning) (2)

- Clasele de echivalenta **nu trebuie sa se suprapuna**, deci orice clase care s-ar suprapune trebuie descompuse in clase separate
- Dupa ce clasele au fost identificate, se alege o valoare din fiecare clasa. In plus, pot fi alese si **date invalide** (care sunt in afara claselor si nu sunt procesate de nici o clasa)
- Alegerea valorilor din fiecare clasa este **arbitrara** deoarece se presupune ca toate valorile vor fi procesate intr-un mod identic

Partitionare de echivalenta - exemplu

- Se testeaza un program care verifica daca un caracter se afla intr-un sir de cel mult 20 de caractere. Mai precis, pentru un intreg *n* aflat intre 1 si 20, se introduc caractere, iar apoi un caracter *c*, care este apoi cautat printre cele *n* caractere introduse anterior. Programul va produce o iesire care va indica **prima pozitie din sir unde a fost gasit caracterul c** sau **un mesaj indicand ca acesta nu a fost gasit**. Utilizatorul are optiunea sa caute un alt caracter tastand **y (yes)** sau sa termine procesul tastand **n (no)**.

Intrari

- un intreg pozitiv ***n***
- un sir de caratere ***x***
- caracterul care se cauta ***c***
- optiune de a cauta sau nu an alt caracter ***s***

Domeniul de intrari:

- *n* trebuie sa fie intre 1 si 20, deci se disting **3 clase de echivalenta**:
  - N_1 = 1..20
  - N_2 = { n | n < 1}
  - N_3 = {n | n > 20}
- intregul *n* determina lungimea sirului de caractere si nu se precizeaza nimic despre tratarea diferita a sirurilor de lungime diferita deci a doua intrare **nu determina clase de echivalenta suplimentare**
- *c* **nu determina clase de echivalenta suplimentare**
- optiunea de a cauta un nou caracter este binara, deci se disting **2 clase de echivalenta**
  - S_1 = {y}
  - S_2 = {n}

Iesiri

- Pozitia la care caracterul se gaseste in sir
- Un mesaj care arata ca nu a fost gasit
- Acestea sunt folosite pentru a imparti domeniul de intrare in **2 clase**: una pentru cazul in care caracterul se afla in sirul de caractere si una pentru cazul in care acesta lipseste
- C_1(x) = { c | c se afla in x}
- C_2(x) = { c | c nu se afla in x}

Clase de echivalenta

- Clasele de echivalenta pentru intregul program se pot obtine ca o **combinatie a claselor individuale**:
- C_111 = { (n, x, c, s) | n \in N_1, |x| = n, c \in C_1(x), s \in S_1}
- C_112 = { (n, x, c, s) | n \in N_1, |x| = n, c \in C_1(x), s \in S_2}
- C_121 = { (n, x, c, s) | n \in N_1, |x| = n, c \in C_2(x), s \in S_1}
- C_122 = { (n, x, c, s) | n \in N_1, |x| = n, c \in C_2(x), s \in S_2}
- C_2 = { (n, x, c, s) | n \in N_2}
- C_3 = { (n, x, c, s) | n \in N_3}

Date de test

- Setul de date de test se alcatuieste alegandu-se **o valoare a intrarilor pentru fiecare clasa de echivalenta**. De exemplu:
- C_111 : (3, abc, a, y)
- C_112 : (3, abc, a, n)
- C_121 : (3, abc, d, y)
- C_122 : (3, abc, d, n)
- C_2 : (0, _, _, _)
- C_3 : (25, _, _, _)

Avantaje

- **Reduce drastic numarul de date de test** doar pe baza specificatiei
- Potrivita pentru aplicatii de tipul procesarii datelor, in care intrarile si iesirile sunt **usor de identificat** si iau valori distincte

Dezavantaje

- Modul de definire al claselor **nu este evident** (depinde de experienta testerului).
- In unele cazuri, desi specificatia ar putea sugera ca un grup de valori sunt procesate identic, **acest lucru nu este adevarat**.
- Mai putin aplicabile pentru situatii cand intrarile si iesirile sunt simple, dar **procesarea este complexa**.

Analiza valorilor de frontiera (boundary value analysis)

- Analiza valorilor de frontiera este folosita de obicei **impreuna cu partitionarea de echivalenta**.
- Se concentreaza pe examinarea valorilor de frontiera ale claselor, care deobicei sunt o **sursa importanta de erori**.

Exemplu

- N_1 = 1..20
  - 1, 20 (**valori de frontiera**) si o valoare din interior
- N_2 = { n | n < 1}
  - 0 (**valoare de frontiera**) si o valoare din interior
- N_3 = {n | n > 20}
  - 21 (**valoare de frontiera**) si o valoare din interior
- C_1(x) = { c | c se afla in x}
  - pe prima pozitie in x, pe ultima pozitie in x, in interiorul lui x
- C_2(x) = { c | c nu se afla in x}
  - **nu exista frontiere clare**, deci nu apar valori suplimentare

Observatie

- Modul de alegere al frontierelor **nu este evident** si depinde de experienta testerului.
- Exemplu: pentru alegerea frontierei poate fi considerat **numarul de aparitii** ale lui c in sirul x.
  - 0 aparitii, 1 aparitie, mai mult de o aparitie

Partitionarea in categorii (category-partition)

- Se bazeaza pe cele doua anterioare.
- Cauta sa genereze date de test care "**acopera**" functionalitatea sistemului si **maximizeaza posibilitatea de gasire a erorilor**.

Partitionarea in categorii - pasi

- Descompune specificatia functionala in **unitati** (programe, functii, etc.) care pot fi testate separat
- Pentru fiecare unitate, identifica **parametrii si conditiile de mediu** (ex. starea sistemului la momentul executiei) de care depinde comportamentul acesteia.
- Gaseste **categoriile** (proprietati sau caracteristici importante) fiecarui parametru sau conditii de mediu.
- Partitioneaza fiecare categorie in **alternative**. O alternativa reprezinta o multime de valori similare pentru o categorie.
- Scrie **specificatia de testare**. Aceasta consta in lista categoriilor si lista alternativelor pentru fiecare categorie.
- Creeaza **cazuri de testare** prin alegerea unei combinatii de alternative din specificatia de testare (fiecare categorie contribuie cu zero sau o alternativa).
- Creeaza **date de test** alegand o singura valoare pentru fiecare alternativa.

Partitionarea in categorii - exemplu

- Descompune specificatia in unitati: avem **o singura unitate**
- Identifica parametrii: *n*, *x*, *c*, *s*
- Gaseste **categorii**:
  - *n* : daca este in intervalul valid 1..20
  - *x* : daca este de lungime minima, maxima sau intermediara
  - *c* : daca ocupa prima sau ultima pozitie sau o pozitie in interiorul lui x sau nu apare in x
  - *s* : daca este pozitiv sau negativ
- Partitioneaza fiecare categorie in **alternative**:
  - *n* : <0, 0, 1, 2..19, 20, 21, >21
  - *x* : lungime minima, maxima sau intermediara
  - *c* : pozitia este prima, in interior, sau ultima sau c nu apare in x
  - *s* : y, n

Partitionarea in categorii – exemplu (cont.)

- Scrie specificatia de testare
  - ***n***
    1) {n | n < 0}
    2) 0
    3) 1
    4) 2..19
    5) 20
    6) 21
    7) {n | n > 21}
  - ***x***
    1) {x | |x| = 1}
    2) {x | 1 < |x| < 20}
    3) {x | |x| = 20}
  - ***c***
    1) { c | c se afla pe prima pozitie in x}
    2) { c | c se afla in interiorul lui x}
    3) { c | c se afla pe ultima pozitie in x}
    4) { c | c nu se afla in x}
  - ***s***
    1) y
    2) n

Cazuri de testare

- Din specificatia de testare ar trebui sa rezulte 7 * 3 * 4 * 2 = **168 de cazuri de testare**.
- Unele combinatii de alternative nu au sens si pot fi **eliminate**.
- Alternativele vor fi combinate doar daca **conditiile de selectie sunt satisfacute**.
- In exemplu, numarul cazurilor de testare se reduce la **24**.

Cazurile de testare - exemplu

- n1
- n2
- n3x1c1s1
- n3x1c1s2
- n3x1c4s1
- n3x1c4s2
- n4x2c1s1
- n4x2c1s2
- n4x2c2s1
- n4x2c2s2
- n4x2c3s1
- n4x2c3s2
- n4x2c4s1
- n4x2c4s2
- n5x3c1s1
- n5x3c1s2
- n5x3c2s1
- n5x3c2s2
- n5x3c3s1
- n5x3c3s2
- n5x3c4s1
- n5x3c4s2
- n6
- n7

Avantaje si dezavantaje

- Pasii de inceput (identificarea parametrilor si a conditiilor de mediu precum si a categoriilor) **nu sunt bine definiti** si se bazeaza pe experienta celui care face testarea.
- Pe de alta parte, odata ce acesti pasi au fost trecuti, aplicarea metodei este **foarte clara**.
- Este **mai clar definita decat metodele functionale anterioare** si poate produce date de testare mai cuprinzatoare, care testeaza functionalitati suplimentare.
- Pe de alta parte, datorita **exploziei combinatorice**, pot rezulta date de test de foarte mare dimensiune

Metoda Grafului Cauza-Efect (Cause-Effect Graphing)

- Partiționarea în categorii poate produce un număr mare de combinații de intrări, dintre care o mare parte poate fi **nefezabilă**.
- Metoda grafului cauza-efect (cunoscuta și ca modelarea dependențelor) se concentrează pe modelarea relațiilor de dependent între **condițiile de intrare** ale programului (**cauze**) și **condițiile de ieșire** (**efecte**).
- Relația dintre acestea este exprimată sub forma unui **graf cauza-efect**.

Graf cauza-efect

- Reprezentare vizuală a relației logice dintre cauze și efecte, exprimabilă ca o **expresie Booleană**.
- **Cauza** = orice conditie in specificatie (cerinte) care poate afecta raspunsul programului.
- **Efect** = raspunsul programului la o combinative de conditii de intrare.
- Efectul **nu este in mod necesar o iesire** (poate fi un mesaj de eroare, un display, o modificare a unei baze de date sau chiar un punct de testare intern)

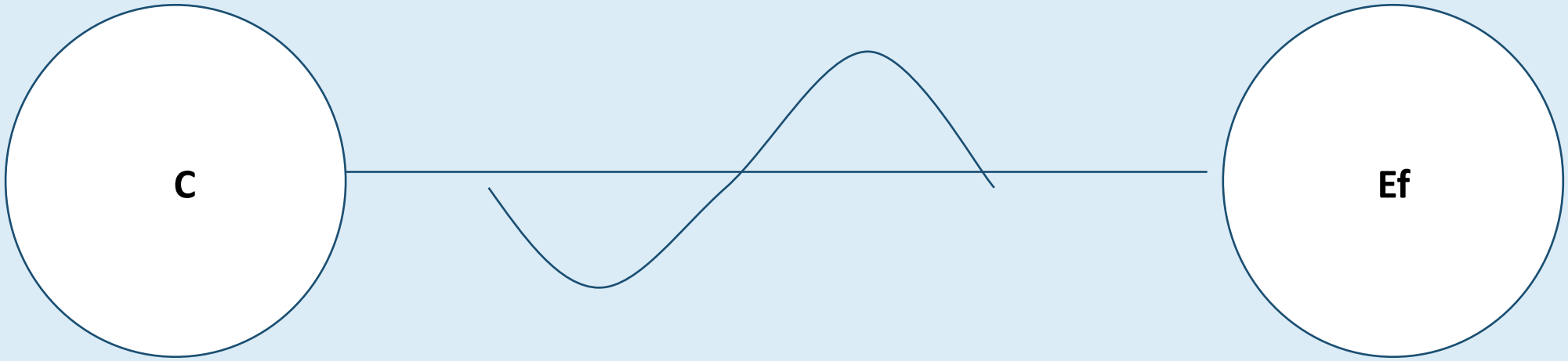
Notatii: Relatii Cauza-Efect (1)

- **Implies** : if C then Ef



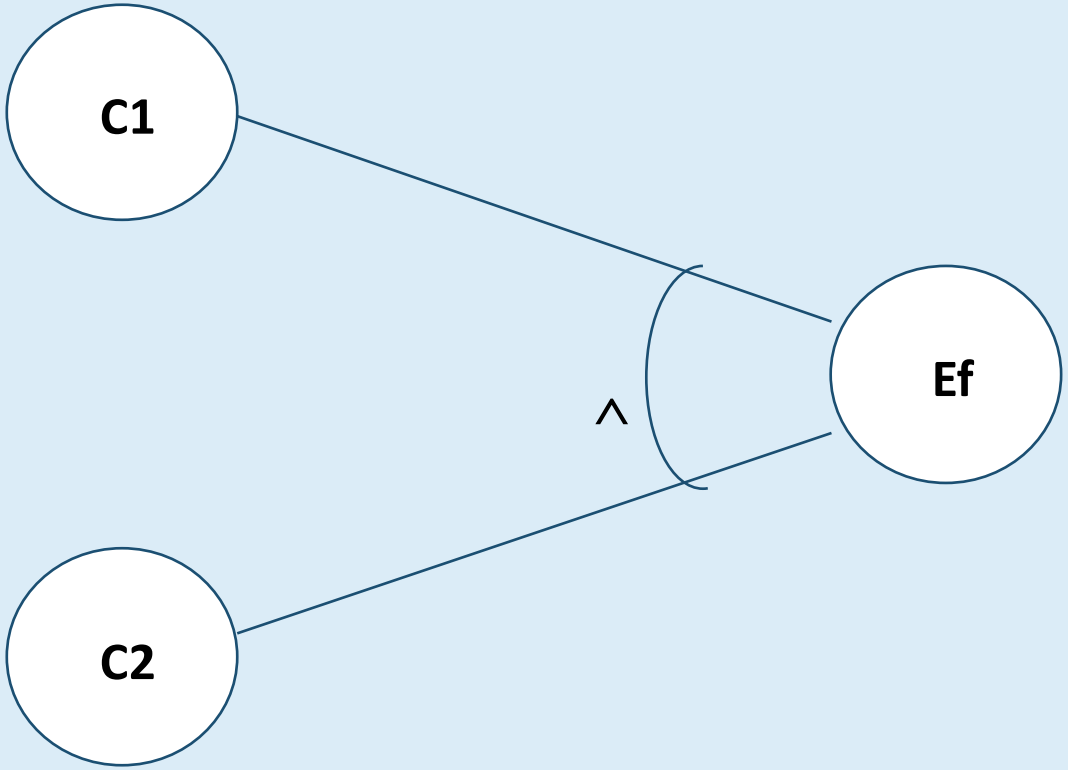
Notatii: Relatii Cauza-Efect (2)

- **Not – Implies** : if (¬C) then Ef



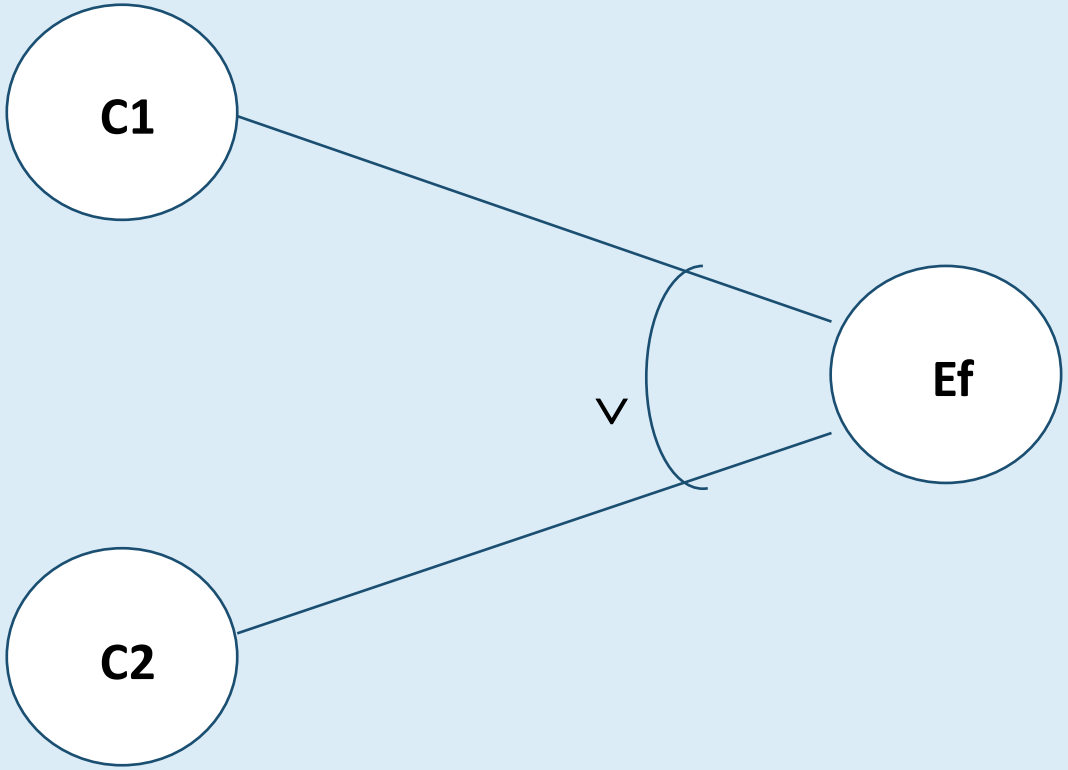
Notatii: Relatii Cauza-Efect (3)

- **And – Implies**: if (C1 && C2) then Ef



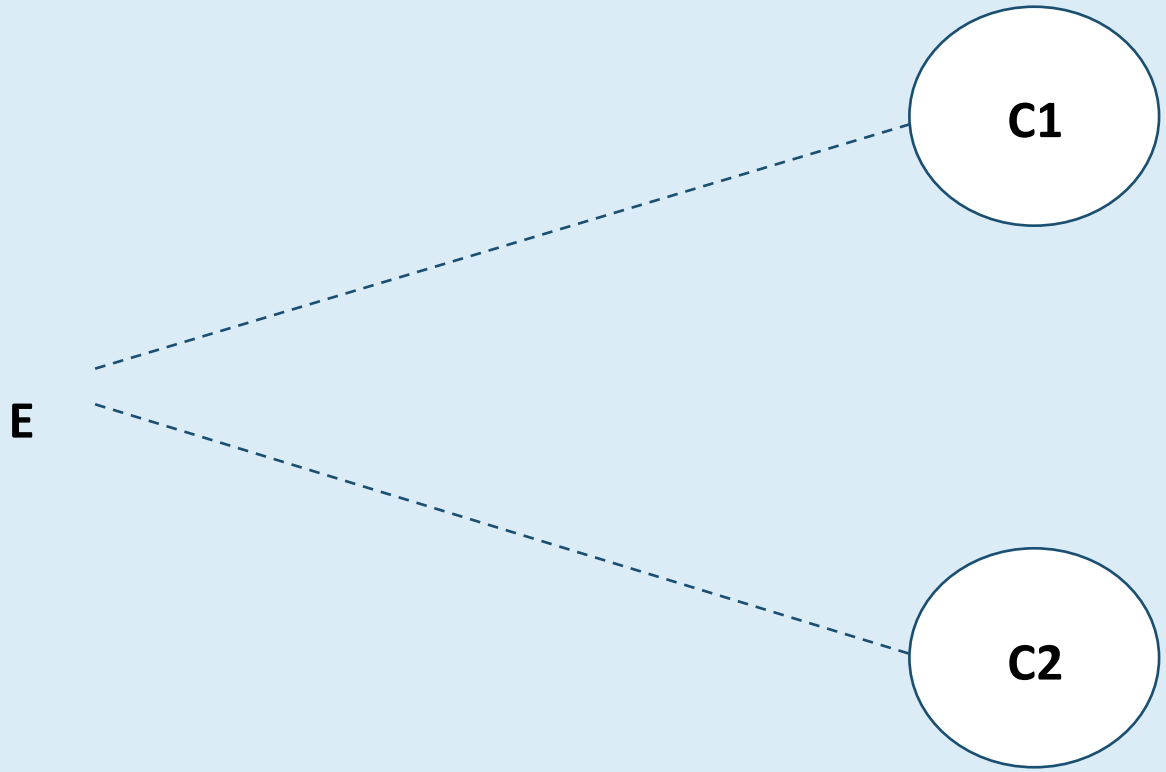
Notatii: Relatii Cauza-Efect (4)

- **Or – Implies**: if (C1 || C2) then Ef



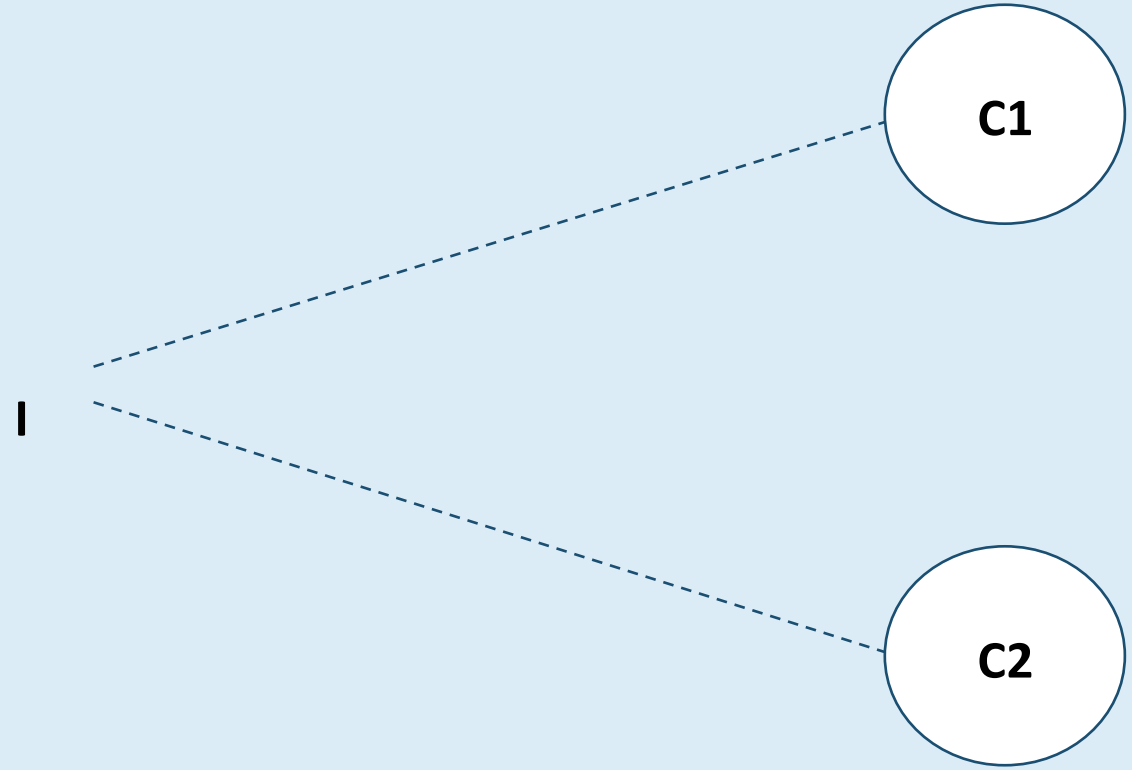
Notatii: Constrangeri intre cauze (1)

- **E (exclusive)**: fie C1 sau C2 (cel mult unul dintre ele)



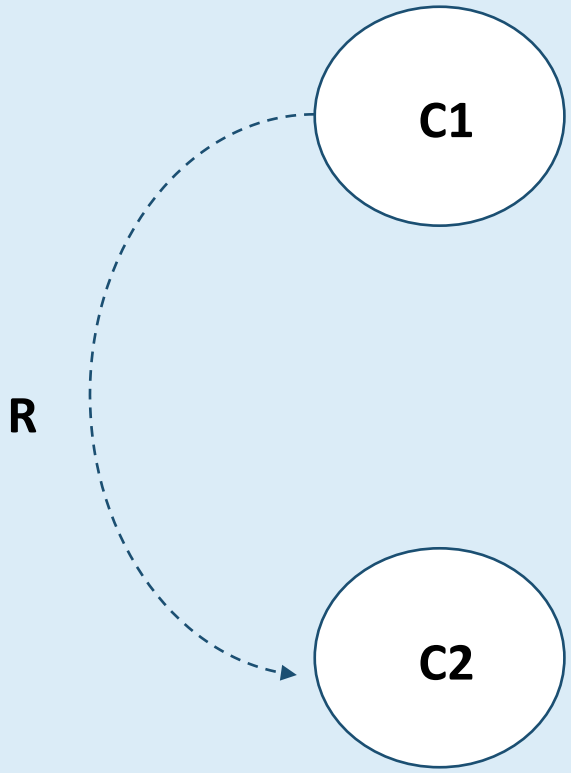
Notatii: Constrangeri intre cauze (2)

- **I (Inclusive)**: cel putin C1 sau C2



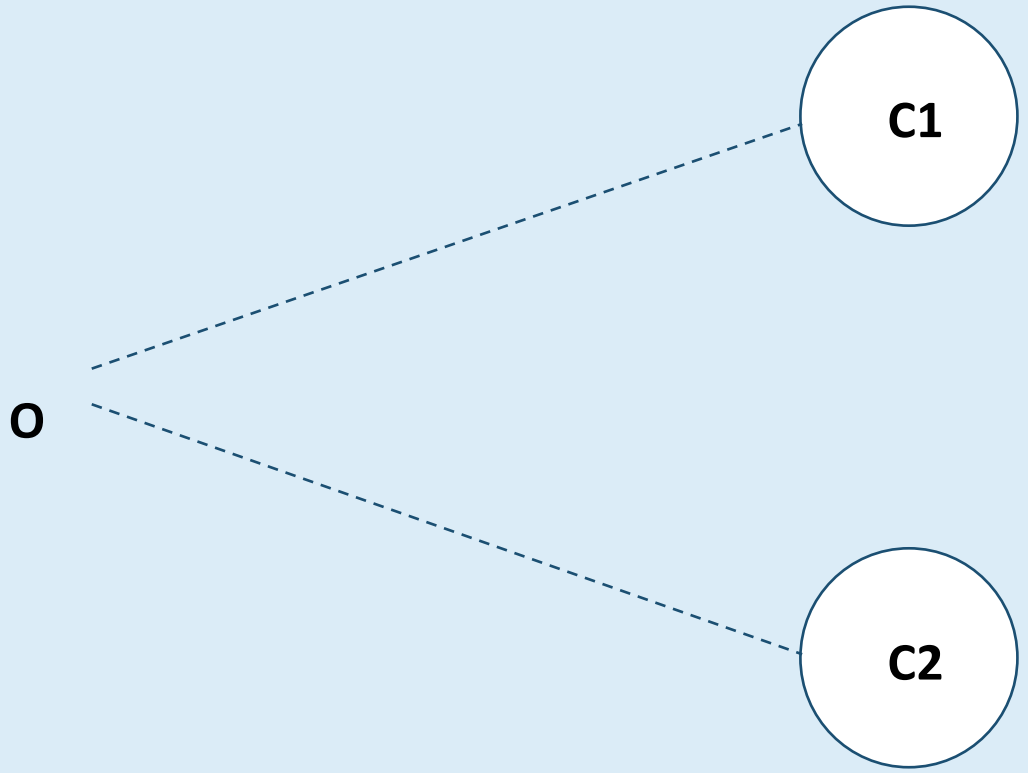
Notatii: Constrangeri intre cauze (3)

- **R (Requires)** C1 cere C2 (daca C1 atunci C2)



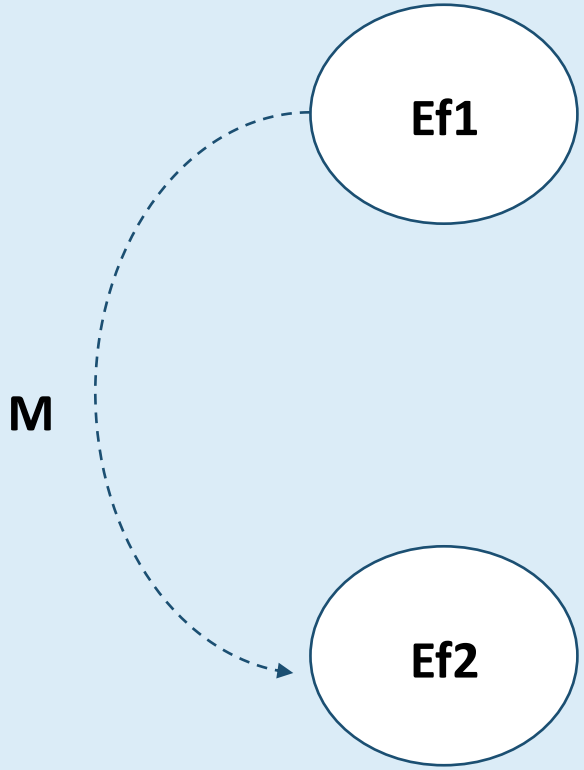
Notatii: Constrangeri intre cauze (4)

- **O (one and only one)**: unul si numai unul dintre C1 si C2



Notatii: Constrangeri intre efecte

- **M (Masks)**: Ef1 mascheaza Ef2 (daca Ef1 atunci Ef2)



Crearea grafului cauza-efect (exemplu)

- O cafenea online permite construirea unei comenzi din **trei categorii** de produse:
  - **Bautura de baza**: Espresso, Cappuccino, Cold Brew
  - **Topping**: sirop de vanilie, caramel drizzle
  - **Pahar**: Medium, Large, cana reutilizabila
- O comanda contine intre 1 si 4 articole, **cel mult cate unul din fiecare categorie**.
- Interfata grafica are trei ferestre (cate una pentru fiecare categorie) si o fereastra in care sunt afisate **cupoanele si cadourile** primite.
- Paharele Medium si Large pot fi alese cu **orice bautura sau separat**.
- Cana reutilizabila poate fi aleasa **doar impreuna cu un Cold Brew**.
- Siropul de vanilie este oferit **cadou** la comanda unui Cappuccino sau a unui Cold Brew.
- Paharele si topping-urile, in afara de cana reutilizabila, pot fi alese **separat**, fara a comanda si o bautura de baza.
- **Recompense**: Espresso -> cupon **10%** reducere; Cappuccino sau Cold Brew -> cupon **20%** reducere; Cold Brew + cana reutilizabila -> cupon **30%** reducere si caramel drizzle cadou.

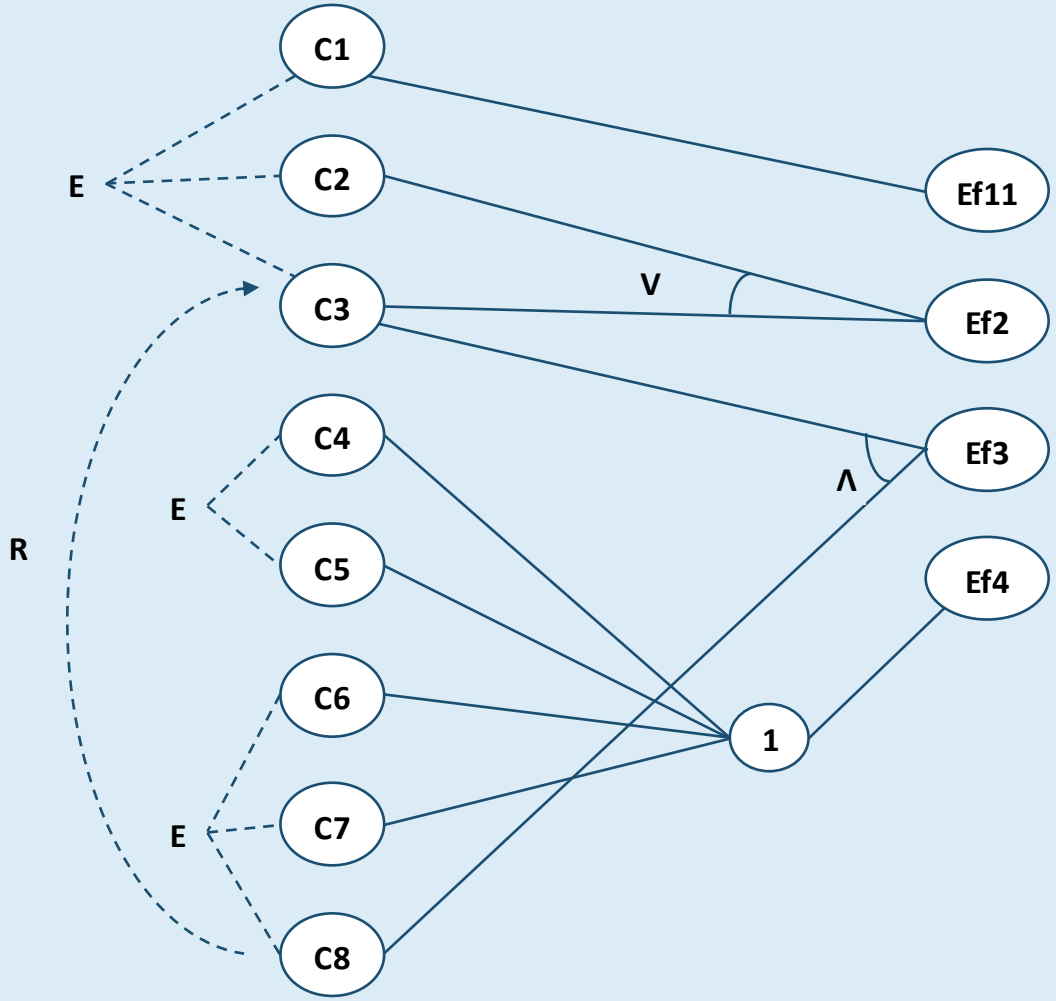
Cauze

- C1: Comanda **Espresso**
- C2: Comanda **Cappuccino**
- C3: Comanda **Cold Brew**
- C4: Comanda **sirop de vanilie**
- C5: Comanda **caramel drizzle**
- C6: Comanda **pahar Medium**
- C7: Comanda **pahar Large**
- C8: Comanda **cana reutilizabila**

Efecte

- Ef1: Cupon **10%** reducere
- Ef2: Cupon **20%** reducere si sirop de vanilie (pot fi considerate efecte separate, dar se complica fara motiv graful)
- Ef3: Cupon **30%** reducere si caramel drizzle
- Ef4: **Nicio reducere**

Graful cauza-efect



Graful cauza-efect pentru exemplul cu cautarea unui caracter

- Spre deosebire de exemplul anterior, aici cauzele si efectele **nu rezulta direct din cerinte**, ci trebuie identificate ca **conditii logice** de care depinde comportamentul programului.
- Cauze - conditiile deja folosite la **partitionarea de echivalenta**, nu optiuni explicite de cumparare:
  - C1: n este in **intervalul valid** (1..20)
  - C2: **n < 1**
  - C3: **n > 20**
  - C4: **c se afla in sirul x**
  - C5: **s = y** (utilizatorul cauta un nou caracter)
- Efecte - comportamentele distincte pe care programul le poate afisa:
  - Ef1: se afiseaza **pozitia** lui c in x
  - Ef2: se afiseaza mesajul **"caracter negasit"**
  - Ef3: se afiseaza **eroare** (n in afara intervalului valid)
  - Ef4: procesul **se repeta** pentru un nou caracter
- **Relatii**: (C1 si C4) implica Ef1; (C1 si non-C4) implica Ef2; non-C1 implica Ef3; C5 implica Ef4.

Crearea tabelului de decizie

- **Input:** Un graf cauza-efect avand cauze C1, ..., Cp si efecte Ef1, ..., Efq.

- **Output**: Un tabel de decizie avand **N = p + q** randuri si **M** coloane, unde M depinde de relatia dintre cauze si efect

Procedura de creare a tabelului de decizie

1. **Initializeaza** nr_coloane = 0 (tabel de decizie gol)
2. For i = 1 to q

   2.1 e = Ef_i (selecteaza urmatorul efect pentru procesare)

   2.2 **Gaseste** combinatiile de conditii care produc aparitia efectului e.

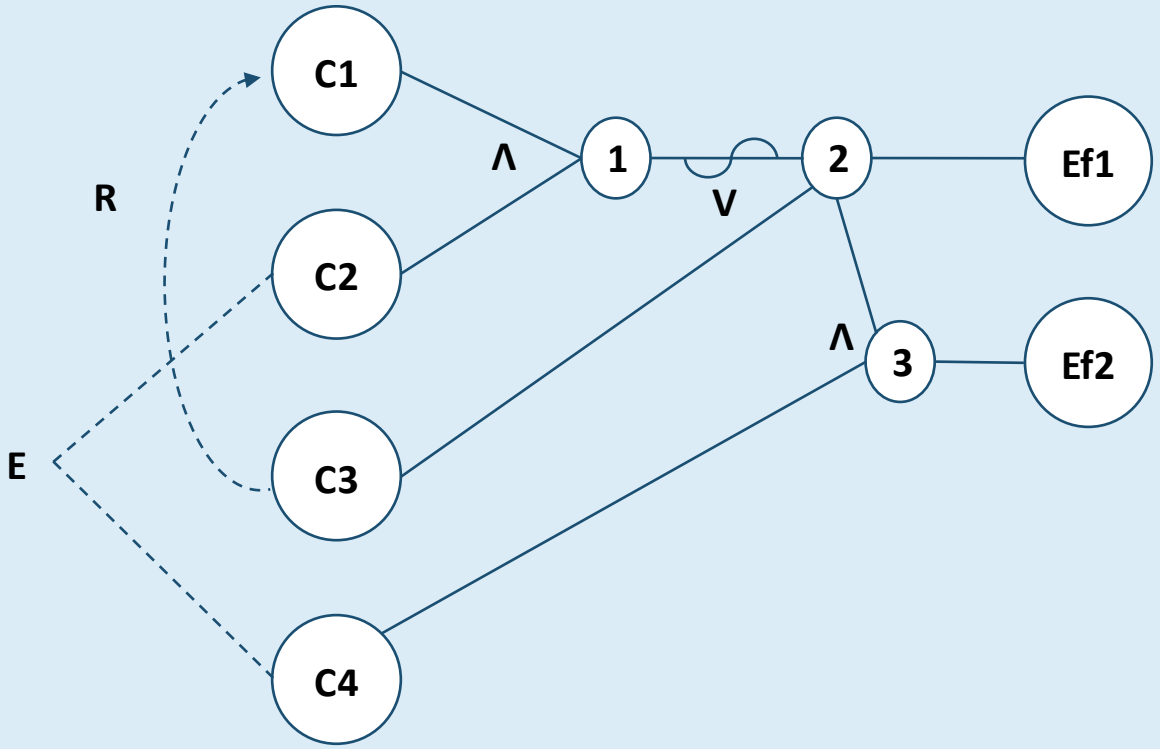
   Fie V1, ..., Vm_i aceste combinatii, $m_i > 0$. **Seteaza** Vk(j), $p < j \leq p+q$, la 1 daca efectul Ef_j apare ca urmare a combinatiei respective si la 0 in caz contrar

   2.3 **Actualizeaza** tabelul de decizie.

   Adauga coloanele V1, ..., Vm_i la tabel incepand cu pozitia nr_coloane + 1.

   2.4 nr_coloane = nr_coloane + m_i.

Crearea tabelului de decizie: exemplu



Crearea tabelului de decizie: procedura (1)

- **Pas 1:** nr_coloane = 0
- **Pas 2:** i = 1
- **Pas 2.1:** e = Ef1

Crearea tabelului de decizie: procedura (2)

- **Pas 2.2:** Se cauta valorile lui C1, C2, C3 astfel incat $\neg (C1 \wedge C2) \vee C3 = 1$

1	0	1
0	1	1
0	0	1
1	1	1
1	0	0
0	1	0
0	0	0

- In plus, se **aplica contrangerea C3 implica C1**

1	0	1
1	1	1
1	0	0
0	1	0
0	0	0

Crearea tabelului de decizie: procedura (3)

- Se adauga C4 = 0 si **valorile corespunzatoare** pentru Ef1 si Ef2

V1	1	0	1	0	1	0
V2	1	1	1	0	1	0
V3	1	0	0	0	1	0
V4	0	1	0	0	1	0
V5	0	0	0	0	1	0

Crearea tabelului de decizie: procedura (4)

- **Pas 2.3:** Matricea obtinuta este transpusa si adaugata la tabelul de decizie incepand cu pozitia nr_coloane + 1 = 1

	1	2	3	4	5
C1	1	1	1	0	0
C2	0	1	0	1	0
C3	1	1	0	0	0
C4	0	0	0	0	0
Ef1	1	1	1	1	1
Ef2	0	0	0	0	0

- **Pas 2.4:** Se actualizeaza nr_coloane = 0 + 5 = 5

Crearea tabelului de decizie: procedura (5)

- **Pas 2:** i = 2
- **Pas 2.1:** e = Ef2

Crearea tabelului de decizie: procedura (6)

- **Pas 2.2:** Se cauta valorile lui C1, C2, C3, C4 astfel incat $(\neg (C1 \wedge C2) \vee C3) \wedge C4 = 1$
- Folosind combinatiile C2, C2, C3 anterioare pentru Ef1, obtinem
- In plus, se aplica constrangerea ca **C2 si C4 sa nu existe simultan**
- Se adauga valorile corespunzatoare pentru Ef1 si Ef2

1	**0**	**1**	**1**
1	1	1	1
1	0	0	1
0	1	0	1
0	0	0	1

1	**0**	**1**	**1**
1	0	0	1
0	0	0	1

V1	**1**	**0**	**1**	**1**	**1**	**1**
V2	1	0	0	1	1	1
V3	0	0	0	1	1	1

Crearea tabelului de decizie: procedura (7)

- **Pas 2.3:** Matricea obtinuta este transpusa si adaugata la tabelul de decizie incepand cu pozitia nr_coloane + 1 = 6

	1	2	3	4	5	6	7	8
C1	1	1	1	0	0	1	1	0
C2	0	1	0	1	0	0	0	0
C3	1	1	0	0	0	1	0	0
C4	0	0	0	0	0	1	1	1
Ef1	1	1	1	1	1	1	1	1
Ef2	0	0	0	0	0	1	1	1

- **Pas 2.4:** Se actualizeaza nr_coloane = 5 + 3 = 8
- **Procedura se termina**. Nr coloane rezultate este **M = 8**.

Generare cazurilor de testare

- Fiecare coloana din tabelul de decizie genereaza **cel putin un caz de testare**, corespunzator combinatiei C1, ..., Cp respective
- **Observatie**: C1, ..., Cp sunt in general expresii care folosesc variabile, etc., deci pentru o combinatie pot fi selectate **mai multe cazuri de testare**
- **Problema**: Explozie a starilor datorita combinatiei de cauze.
- **Solutie**: Limitarea numarului de cazuri de testare folosind **euristici**